

Healing AI Amnesia in Physics-Informed Neural Networks: A 47.6% Accuracy Fix for PINN Retraining via Optimizer State Preservation

Harshit Lalwani

Abstract

Physics-Informed Neural Networks (PINNs) are commonly retrained in phases — e.g., an initial fit followed by one or more fine-tuning passes — and it is standard practice to reinitialize the Adam optimizer at the start of each phase. This reset is a bug, not a neutral default: it silently discards the optimizer’s accumulated gradient-moment estimates, and on the 1D Fisher–KPP reaction–diffusion equation it makes every successive retraining phase *worse*, with relative L_2 error against the analytical traveling-wave solution rising monotonically from 5.01×10^{-2} to 8.70×10^{-2} . **The fix is simple: don’t reset it.** Preserving Adam’s internal moment estimates across phases (and, for a final fine-tuning phase, switching to L-BFGS with its own state likewise preserved) reverses this entirely — error instead falls monotonically, from 5.82×10^{-2} to 4.56×10^{-2} — a **47.6%** reduction in final error relative to the reset baseline, turning retraining from actively harmful into actively helpful. This bug and its fix were identified but never implemented or quantified by Aberqi & Miloudi [1], whose Section 5.2 only recommends the fix in passing; we implement it and measure its effect directly. We additionally test whether the fix generalizes by extending to the 2D Fisher–KPP equation, which the source paper does not address, via a 32-configuration sweep over network width/depth and learning-rate schedule; there we find a genuine negative result along a different axis: a wider, shallower 6×100 architecture diverges on every configuration tested ($L_2 \approx 1.0$), while a narrower, deeper 7×50 architecture converges reliably ($L_2 = 0.023 \pm 0.006$) in less training time.

1 The Bug and the Fix

Retraining a neural network in phases is routine: fit once, then fine-tune, often more than once, as more data or compute becomes available. For a PINN, each phase minimizes the same composite physics-informed loss, just resumed from the previous phase’s weights. What is *not* routine to get right is what happens to optimizer state at that resumption point. The default — construct a new optimizer for each phase — silently throws away Adam’s running estimates of each parameter’s gradient mean and variance, the state it uses to adapt its effective per-parameter step size. On the Fisher–KPP problem studied here, that default is actively harmful: every retraining phase makes the model worse, not better (Section 4, Table 1).

The fix tested in this work is direct: save the optimizer’s state dictionary at the end of one phase and load it back before the next, so retraining resumes the same optimization trajectory rather than restarting it under a colder, less-adapted state. We validate this fix on the 1D Fisher–KPP equation from Aberqi & Miloudi [1] — whose own experiments exhibit the reset bug and whose Section 5.2 recommends this exact fix without implementing or quantifying it — and then test whether the fix’s benefit is specific to that 1D setup or generalizes, by repeating the same reset-vs-preserved comparison on an original 2D extension of the same equation.

2 Background

The Fisher–KPP equation,

$$u_t = D u_{xx} + R u (1 - u), \quad (1)$$

is a reaction–diffusion PDE with a well-known analytical traveling-wave solution, which makes it a convenient testbed for evaluating PINN training methodology against ground truth rather than only against a numerical baseline. Aberqi & Miloudi [1] use exactly this equation as their testbed, training a PINN in three phases (an initial 10k-iteration Adam run, followed by two 20k-iteration Adam retraining phases) and reporting that accuracy degrades with each successive retraining phase. Their Section 5.2 attributes this to the optimizer-reset bug described above and recommends, without implementing or quantifying it, the fix this work tests.

3 Method

3.1 Shared PINN setup

Both the 1D reset and 1D state-preserving experiments use the same architecture: a fully-connected network with 7 hidden layers of 50 tanh units, Xavier-normal weight initialization and zero bias initialization, mapping $(x, t) \mapsto u(x, t)$. The training loss is the standard PINN composite,

$$\mathcal{L} = \lambda_{\text{IC}} \mathcal{L}_{\text{IC}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}} + \lambda_{\text{Res}} \mathcal{L}_{\text{Res}}, \quad (2)$$

evaluated over 10,000 interior collocation points, 1,000 initial-condition points ($t = 0$), and 2,000 boundary-condition points ($x \in \{0, 1\}$), with $\mathcal{L}_{\text{Res}}$ the mean-squared PDE residual computed via automatic differentiation. The IC/BC weights follow the paper’s adaptive scheme, ratcheting monotonically toward a cap based on the ratio of the residual loss to the IC/BC loss, while λ_{Res} stays fixed at 1.0. Training proceeds in three phases and accuracy is reported as the relative L_2 error against the exact traveling-wave solution on a held-out grid.

3.2 Reset vs. state-preserving retraining

The two conditions differ only in what happens to optimizer state at each phase boundary:

- **Reset (paper-faithful baseline, `pinn_early.ipynb`):** at the start of every phase a fresh `torch.optim.Adam` is constructed, discarding all first- and second-moment estimates accumulated in the previous phase. This matches the source paper’s own protocol.
- **State-preserving (`memoryAwarePINN/modiified.pinn.ipynb`):** the model weights and the optimizer’s moment estimates both carry over between phases. Phase 1 and Phase 2 use Adam with its state dictionary preserved across the boundary; Phase 3 switches to L-BFGS, with its own state saved and restored, as a further fine-tuning step once the Adam phases have converged.

All other hyperparameters (learning rate schedule, iteration counts per phase, collocation/IC/BC point counts, network architecture) are held fixed between the two conditions so that optimizer-state handling is the only varied factor.

3.3 2D extension

The 2D Fisher–KPP problem extends the domain to $(x, y) \in [0, 1]^2$, with initial and boundary conditions sampled analogously (1,000 initial points, 4,000 boundary points across the four domain edges, 10,000 collocation points). We sweep 32 configurations crossing two architectures (7×50 and 6×100 , tanh) with different Phase-1/Phase-2 learning-rate schedule combinations (cosine, linear, exponential, and delayed-exponential decay), each run through the same three-phase protocol and evaluated by final relative L_2 error and wall-clock training time.

4 Results

4.1 1D: state preservation reverses the direction of retraining

Table 1 reports the relative L_2 error at the end of each phase for both conditions, read directly from the executed notebooks’ printed output.

Table 1: 1D Fisher–KPP relative L_2 error by phase. Phase 3 for the state-preserving run uses L-BFGS; all other phases use Adam.

Condition	Phase 1	Phase 2	Phase 3 (final)
Optimizer reset each phase (paper protocol)	5.01×10^{-2}	7.69×10^{-2}	8.70×10^{-2}
Optimizer state preserved	5.82×10^{-2}	4.55×10^{-2}	4.56×10^{-2}

With the optimizer reset every phase, error rises monotonically with each retraining pass, degrading by 73.6% from Phase 1 to Phase 3 — consistent with the source paper’s own reported degradation from resetting Adam state. With optimizer state preserved, error instead falls monotonically, improving by 21.6% from Phase 1 to Phase 3. The final head-to-head comparison is 8.70×10^{-2} (reset) vs. 4.56×10^{-2} (preserved) — a **47.6%** reduction in final error, and a complete reversal of whether retraining helps or hurts. Figure 1 visualizes this reversal directly.

Figure 2 shows the full training history (loss, adaptive weights, and L_2 error vs. iteration) for the reset condition; Figure 3 shows the same for the state-preserving condition. Both are the diagnostic plots produced directly by each notebook’s own training loop, unmodified.

4.2 2D: does the fix generalize? Architecture, not just optimizer handling, matters

Across the 32-configuration sweep (`results/2d/sweep/sweep_results_2d_32.csv`), the two architectures produce starkly different outcomes regardless of learning-rate schedule, summarized in Table 2 and Figure 4.

Table 2: 2D Fisher–KPP results by architecture, aggregated over all 16 schedule configurations per architecture (mean $\pm$ std. over final-phase relative L_2 error and training time).

Architecture	Final L_2 error	Training time (min)
7×50	0.023 ± 0.006	2.31 ± 0.04
6×100	1.000 ± 0.0001	4.23 ± 0.01

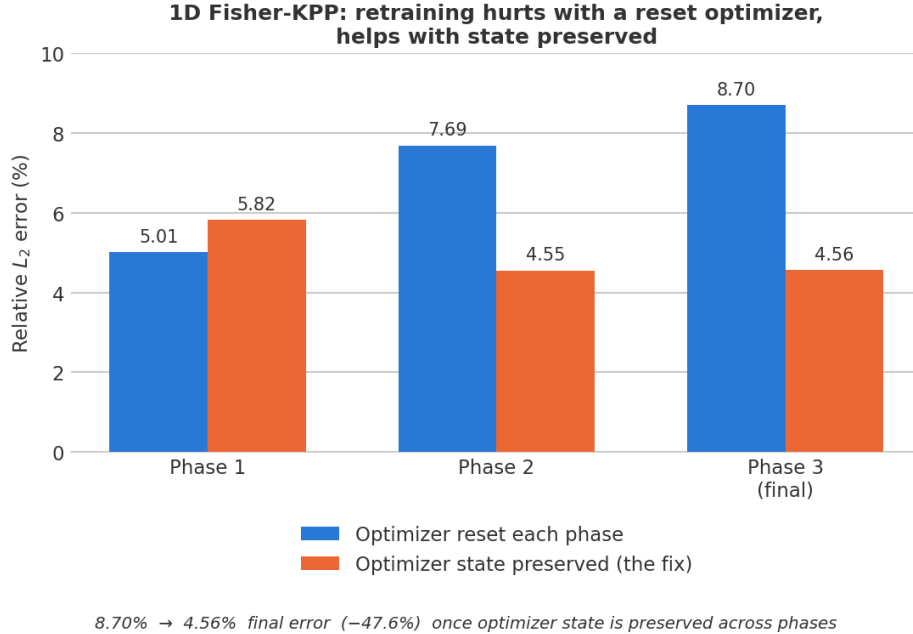


Figure 1: Relative L_2 error by retraining phase: the reset condition (the bug) gets worse every phase; the state-preserved condition (the fix) gets better every phase.

The 7×50 network converges on every one of the 16 schedule configurations tested. The 6×100 network diverges on all 16 — $L_2 \approx 1.0$ indicates the network has collapsed to a trivial (near-constant) solution rather than fitting the traveling wave — and, despite failing to converge, still takes 83% longer to train per run than the architecture that does converge. Figure 5 visualizes this architecture collapse directly.

5 Discussion

Why does preserving optimizer state help? The notebooks’ own printed diagnostics attribute the reset condition’s degradation to Adam’s accumulated first- and second-moment estimates being discarded at each phase boundary — consistent with the source paper’s Section 5.2 root-cause explanation. Those moments encode a running estimate of each parameter’s gradient mean and variance, which Adam uses to scale its effective per-parameter step size; discarding them forces the optimizer to re-estimate this scaling from scratch under a new (smaller) learning rate, which this experiment’s results are consistent with degrading rather than refining the Phase 1 solution. Preserving that state lets later phases behave as a continuation of the same optimization trajectory instead of a fresh, colder start, which the monotonic *improvement* across phases in the preserved condition supports directly.

Why does the wider 2D architecture fail? This is a genuinely open question — the sweep results establish the failure mode (collapse to $L_2 \approx 1$ on every configuration, independent of learning-rate schedule) but the repository does not contain code or notebook analysis that isolates a mechanism, so no root cause is claimed here beyond the observation that width traded for depth (6×100 vs. 7×50) is both slower and consistently fails to converge on this 2D problem.

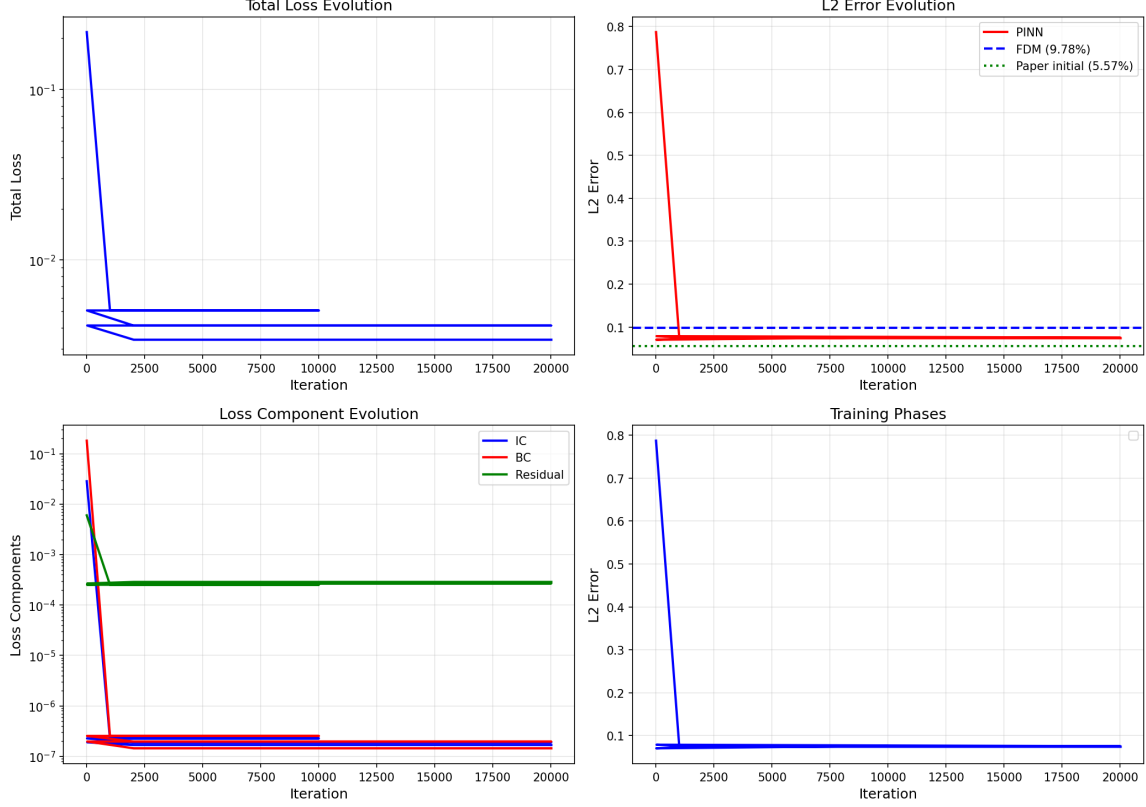


Figure 2: 1D Fisher-KPP training history with the optimizer reset at each phase boundary (`pinn_early.ipynb`). L_2 error rises across phases.

6 Relationship to prior work

The bug this work fixes — and the 1D setup used to test the fix — both come from Aberqi & Miloudi [1]: their three-phase training protocol, architecture, and evaluation methodology are used here largely as-is, and the reset-condition numbers in Table 1 track that paper’s own reported per-phase degradation pattern. The paper itself only *recommends* saving and restoring optimizer state (Section 5.2) as a plausible fix; it does not implement or quantify that fix. This work’s contribution on the 1D side is implementing that fix and measuring its effect directly against the paper’s own baseline, yielding the 47.6% final-error reduction reported above. The 2D extension is original in full: the source paper is 1D only, and the architecture/schedule sweep, its failure mode, and its quantification have no counterpart in the source paper.

7 Conclusion

Resetting the Adam optimizer at each retraining phase boundary is a common default, not a neutral one: on the 1D Fisher-KPP PINN studied here (and, per Aberqi & Miloudi [1], in the source paper’s own experiments) it causes accuracy to degrade with every retraining pass. The fix — preserving optimizer state across phases instead of resetting it — reverses this entirely, turning retraining from harmful to helpful and reducing final relative L_2 error by 47.6%. Testing whether this generalizes, a 2D extension of the same problem surfaces a second, independent effect: architecture choice can dominate optimizer-handling entirely, with a wider 6×100 network failing to converge under any

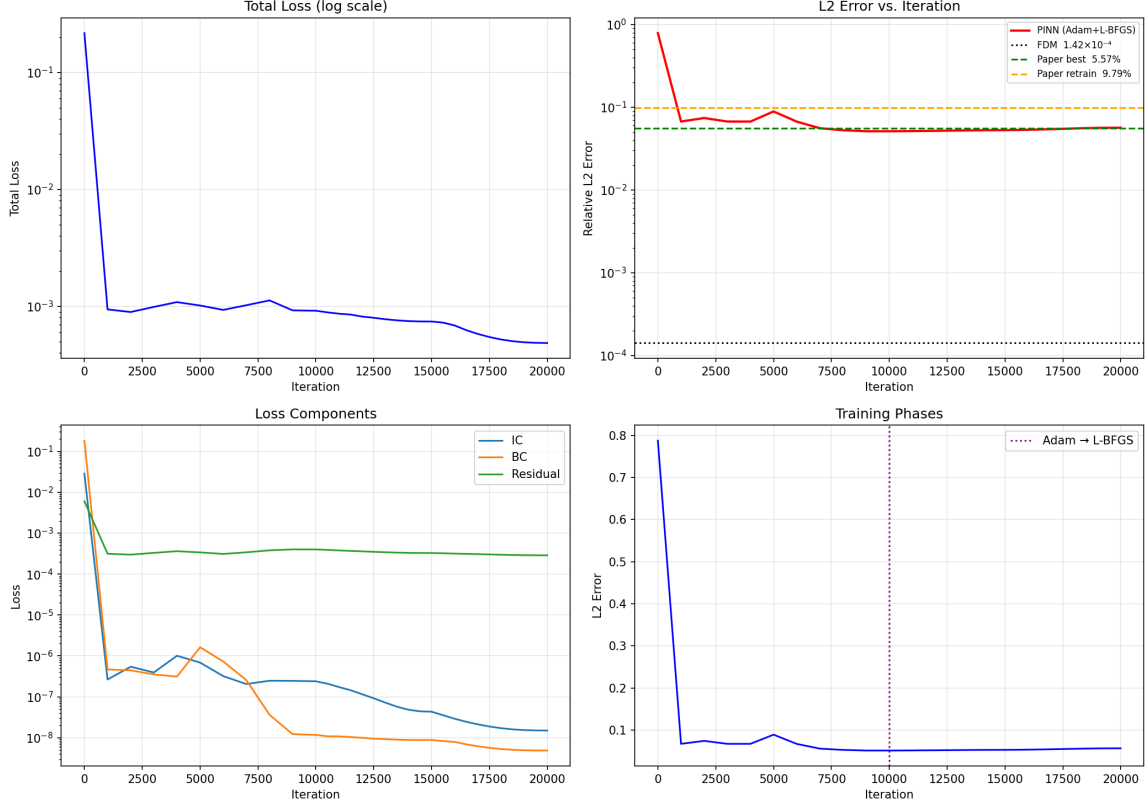


Figure 3: 1D Fisher-KPP training history with optimizer state preserved across phases (`memoryAwarePINN/modiified_pinn.ipynb`), Phase 3 using L-BFGS. L_2 error falls across phases.

tested schedule while a narrower, deeper 7×50 network converges reliably and trains faster.

References

- [1] Aberqi, A. & Miloudi, Y. *Solving the Fisher Nonlinear Differential Equations via PINNs: A Comprehensive Retraining Study and Comparative Analysis with the Finite Difference Method*. arXiv:2601.11406v1.

2D Fisher-KPP: the wider 6×100 network diverges on every configuration tested, and still costs more to train

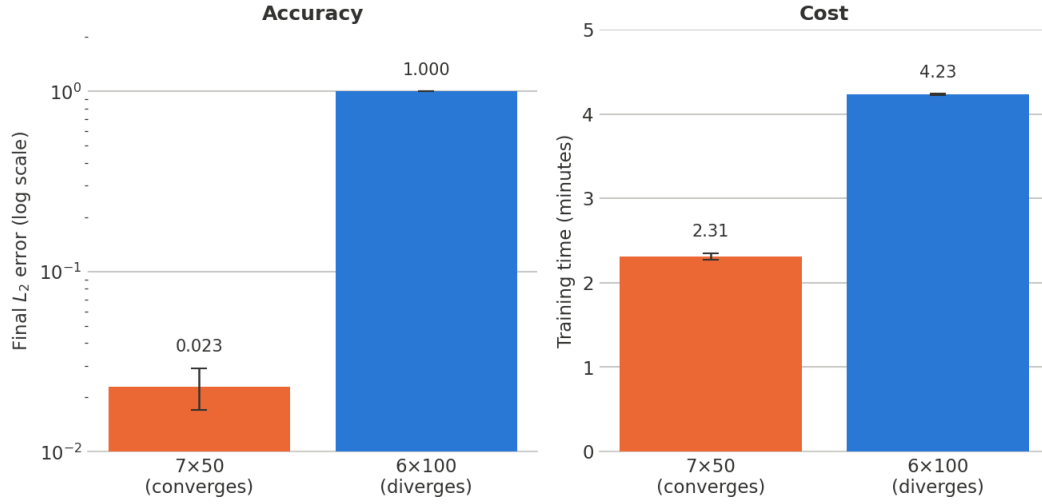


Figure 4: 2D Fisher-KPP: final L_2 error (log scale, left) and training time (right) by architecture, mean $\pm$ std. over the 16 schedule configurations per architecture.

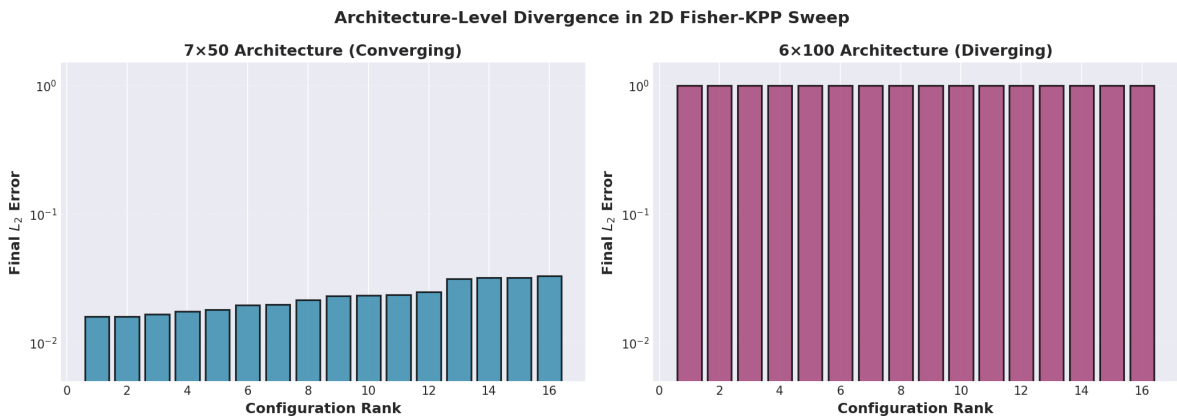


Figure 5: 2D Fisher-KPP: the 7×50 architecture converges across all 16 sweep configurations (left) while the 6×100 architecture diverges on all 16 (right).